ДЕРЖАВНИЙ УНІВЕРСИТЕТ ІНФОРМАЦІЙНО-КОМУНІКАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра технологій цифрового розвитку

ЛАБОРАТОРНА РОБОТА №3

Тема: Успадкування, поліморфізм, принцип LSP
Проєктування ієрархії з поліморфною обробкою

Дисципліна: Конструювання програмного забезпечення Java

Виконав: студент групи ТЦР-31
Бондаренко Олег

Київ — 2026

1. Мета та варіант роботи

Мета роботи — сформувати навички проєктування ієрархії класів із поліморфною обробкою через базовий тип та застосувати принцип підстановки Лісков (LSP). Обрано варіант 4: базовий тип «Сповіщення», різновиди Email, SMS, Push, спільна операція — формування тексту.

2. Структура ієрархії

Абстрактний клас Notification містить спільні дані recipient, перевірку одержувача та метод getRecipient(). Метод formatMessage(String message) оголошений абстрактним, оскільки спосіб формування повідомлення залежить від каналу.

Кожен різновид має власне специфічне поле: subject, sender або application — і перевизначає formatMessage. Конструктор кожного нащадка викликає super(recipient), тому спільна частина об’єкта ініціалізується централізовано без дублювання перевірок.

3. Ключові фрагменти реалізації

```
public abstract class Notification {
    private final String recipient;

    protected Notification(String recipient) {
        if (recipient == null || recipient.isBlank())
            throw new IllegalArgumentException();
        this.recipient = recipient;
    }

    public abstract String formatMessage(String message);
}
```

```
Notification[] notifications = {
    new EmailNotification(...),
    new SmsNotification(...),
    new PushNotification(...)
};

for (Notification notification : notifications) {
    System.out.println(notification.formatMessage(message));
}
```

4. Поліморфна обробка

Масив і цикл працюють лише з базовим типом Notification. Перевірки instanceof та розгалуження за конкретними типами відсутні. Виклик formatMessage динамічно обирає реалізацію відповідного класу-нащадка.

Якщо додати, наприклад, TelegramNotification extends Notification, код циклу змінювати не потрібно. Достатньо реалізувати конструктор, викликати super(recipient) і перевизначити formatMessage.

5. Обґрунтування принципу LSP

Кожен нащадок можна передати туди, де очікується Notification, оскільки він має всі операції базового типу. Жоден нащадок не послаблює доступні можливості, не змінює контракт конструктора базової частини та не викидає несподіваних винятків для коректного повідомлення. Метод повертає рядок сформованого тексту для будь-якого підтримуваного каналу, тому очікувана поведінка клієнта зберігається.

6. Відповіді на контрольні питання

1. Обробка через базовий тип прибирає залежність клієнта від конкретних класів і дозволяє додавати нові різновиди без переписування алгоритму.

2. Спільний контракт оголошують у базовому типі, а відмінності реалізують через перевизначення методів у нащадках. Клієнт працює з базовим типом.

3. LSP означає, що об’єкт нащадка можна використовувати замість об’єкта базового типу без порушення очікуваного результату. Це перевіряють тестуванням клієнтського коду на всіх різновидах через посилання базового типу.

4. У базовий клас виносять спільні поля, інваріанти, перевірки та поведінку. Специфічні дані й алгоритми залишають у похідних класах.

7. Висновок

Створено ієрархію сповіщень із трьома різновидами. Поліморфний масив демонструє єдиний спосіб обробки, а використання super забезпечує коректну ініціалізацію базової частини. Архітектура відповідає принципу LSP і допускає розширення без зміни коду клієнта.

8. Посилання на репозиторій

https://github.com/emil720a1/java-pz3